

Istruzioni. Durante lo svolgimento del test, attenersi rigorosamente alle seguenti istruzioni.

- Gli esercizi devono essere consegnati attraverso il sito Moodle, al seguente indirizzo:

<https://elearning.unica.it/course/view.php?id=188>

- Ogni esercizio deve essere consegnato su un file con estensione `.ml`. Non sono ammesse altre modalità di consegna.
- Non è possibile consegnare alcun esercizio dopo la scadenza stabilita. È possibile re-inviare ogni esercizio un numero arbitrario di volte entro la scadenza stabilita, senza subire penalizzazioni.
- Non è ammesso alcun tipo di collaborazione.
- Se viene richiesto che una funzione abbia un nome specifico, la funzione deve avere quel nome.
- Il file contenente la soluzione non deve contenere esempi (anche commentati).
- Nelle soluzioni degli esercizi 7 e 8 il tipo deve essere contenuto nel file.

1. Scrivere una funzione `foo` con il seguente tipo:

`foo: (int -> 'a) -> 'a -> int`

2. Si consideri la successione $(a_n, b_n)_{n \in \mathbb{N}}$ di coppie di numeri interi data da:

$$\begin{aligned} a_0 &= 0 & b_0 &= 1 \\ a_{n+1} &= \begin{cases} a_n - b_n & \text{se } b_n < a_n \\ a_n + b_n & \text{altrimenti} \end{cases} & b_{n+1} &= \begin{cases} b_n - a_n & \text{se } a_n < b_n \\ b_n + 1 & \text{altrimenti} \end{cases} \end{aligned}$$

Scrivere una funzione `seq: int -> (int * int)` tale che l'espressione `seq n` valuta alla coppia (a_n, b_n) . Nota: è consentito definire funzioni ausiliarie.

3. Date due funzioni $f, g: \mathbb{Z} \rightarrow \mathbb{Z}$, si definisca la funzione $f \circ g: \mathbb{Z} \rightarrow \mathbb{Z}$ come segue:

$$(f \circ g)(x) = \begin{cases} g(x) - f(x) & \text{se } g(x) > f(x) > 0 \text{ oppure se } g(x) < f(x) < 0 \\ g(x) + f(x) & \text{altrimenti} \end{cases}$$

Definire una funzione `comp` con il seguente tipo:

`comp: (int -> int) -> (int -> int) -> (int -> int)`

e tale che `comp f g = f ∘ g`.

4. Scrivere una funzione `foo` con il seguente tipo:

```
foo: 'a list -> ('a -> 'b) -> 'b * 'b
```

5. Definire una funzione `foo1` con tipo:

```
'a list -> 'a list -> int -> 'a list
```

in modo che `foo1 l l' n` sia la lista ottenuta prendendo i primi `n` elementi di `l`, e nel caso in cui `l` abbia meno di `n` elementi, prendendone da `l'` fino a raggiungerne `n`. Ad esempio,

```
foo1 [1;2;3;4;5] [6;7;8;9;10] 3 = [1; 2; 3]
```

```
foo1 [1;2;3;4;5] [6;7;8;9;10] 7 = [1; 2; 3; 4; 5; 6; 7]
```

6. Scrivere una funzione `foo` con il seguente tipo:

```
foo: 'a list -> 'b list -> 'b list * 'a list
```

7. Si consideri il seguente tipo:

```
type 'a tree = Empty | Node of 'a * 'a tree * 'a tree
```

Definire una funzione `sumLeaf` con tipo:

```
'int tree -> int
```

in modo che `sumLeaf t` restituisca la somma dei valori delle foglie dell'albero `t`. Ad esempio:

```
# let t1 = Node (1, Node(2, Node( 3, Empty, Empty), Node(4, Empty, Empty)),  
Empty);;  
  
# sumLeaf t1;;  
  
- : int = 7
```

8. Si consideri la seguente definizione di tipo:

```
type 'a exp = Pip  
           | Top  
           | Plu of int * ('a exp * 'a exp)  
           | Pap of bool * ('a exp * 'a exp) ;;
```

Definire una funzione `eval` che assegni ad un'espressione di tipo `exp` un valore di tipo `int`, interpretando `Pip` e `Top` rispettivamente con 1 e 0, `Plu of int * ('a exp * 'a exp)` con il valore della prima espressione della coppia se l'intero è pari, con il valore della seconda altrimenti, e `Pap of bool * ('a exp * 'a exp)` con il valore della prima espressione della coppia se il valore booleano è vero, con il valore della seconda altrimenti.